

Interacțiunea program C - SO

CURS SO NR. 2/2

Programarea calculatoarelor

Limbajul nativ al calculatorului

- **Limbaj mașină** (cod obiect) - șabloane de numere binare – mapare 1-la-1 cu instrucțiuni de asamblare
- Dependente de platforma (de arhitectura sistemului)

Alte limbaje trebuie **convertite în cod mașină**

- Folosind compilatoare - limbaj compilat, sau
- Folosind interpretoare – limbaj interpretat
- Limbaje **low-level** și **high-level**
 - Apropierea față de limbajul mașină

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

Instrucțiuni în
limbajul C

```
swap:
    muli $2, $5, 4
    add  $2, $4, $2
    lw   $15, 0($2)
    lw   $16, 4($2)
    sw   $16, 0($2)
    sw   $15, 4($2)
    jr   $31
```

Instrucțiuni în
limbaj de asamblare

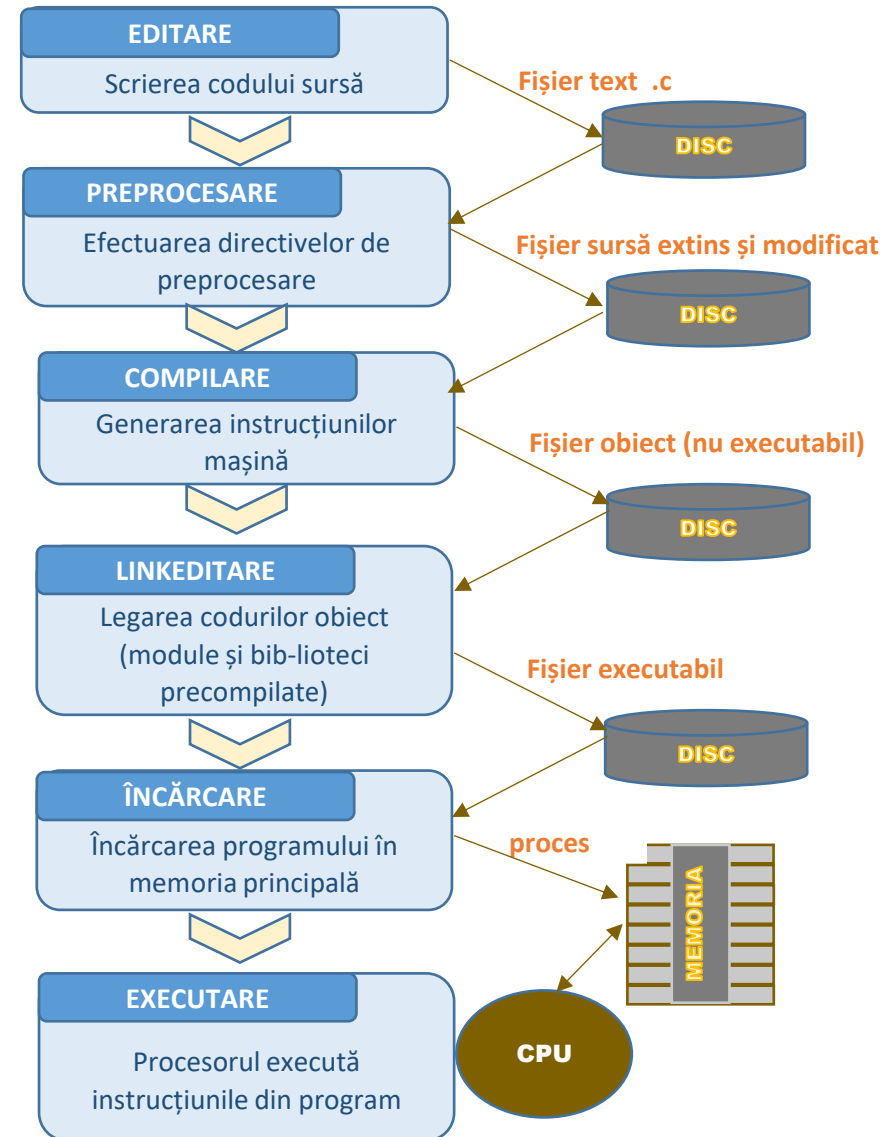
```
0000000010100001000000000011000
00000000000110000001100000100001
1000110001100010000000000000000
10001100111100100000000000000100
1010110011110010000000000000000
10101100011000100000000000000100
0000001111100000000000000001000
```

Instrucțiuni în limbaj mașină

De la cod sursă la program executabil

Etape

- **Editarea codului sursă**
 - Scrierea instrucțiunilor în limbajul de programare și
 - Salvarea fișierului cu extensia .c
- **Preprocesarea**
 - Efectuarea directivelor de preprocesare
 - Ca un editor – modifică și adaugă la codul sursă
- **Compilarea**
 - Verificarea sintaxei
 - Transformare în cod obiect (limbaj mașină) cu extensia .obj
 - Nu este încă executabil !
- **Linkeditarea (editarea legăturilor)**
 - Combinarea codului obiect cu alte coduri obiect
 - Al bibliotecilor asociate fișierelor header incluse
 - Al celorlalte fișiere sursă din program (modular)
 - Transformarea adreselor simbolice în adrese relocabile



Compilare și executare

Compilare

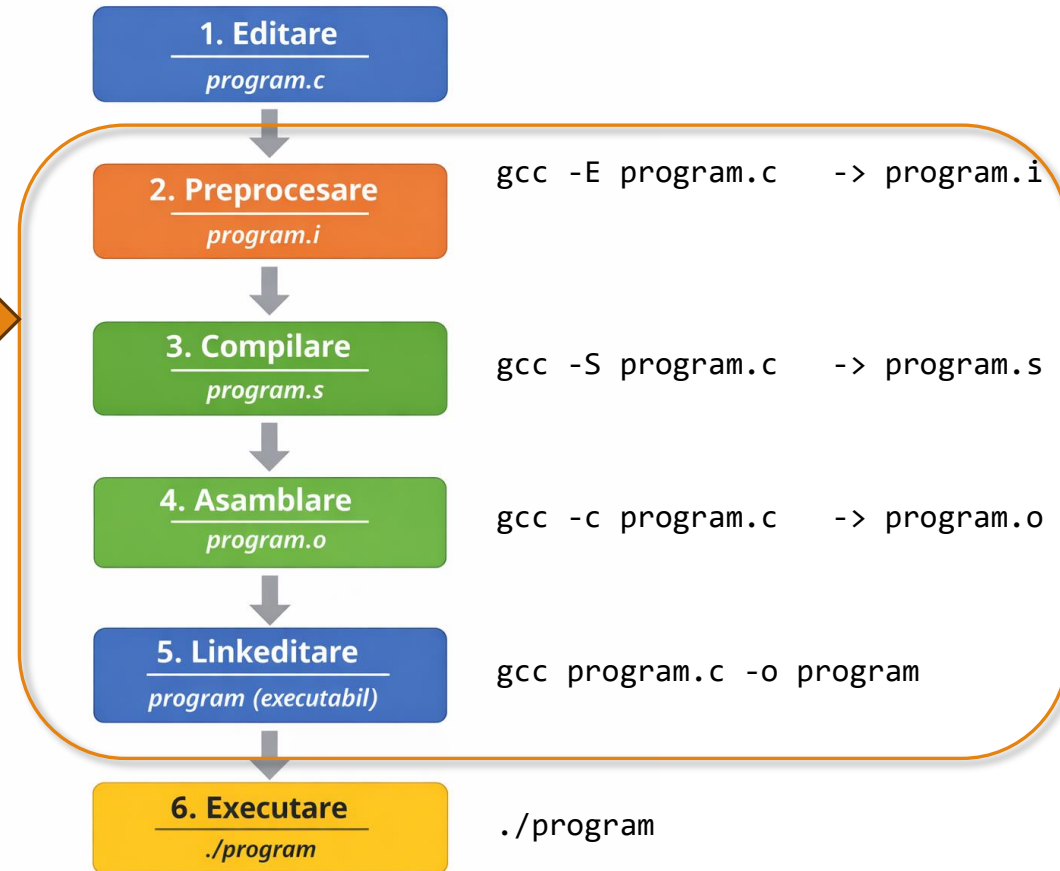
- `gcc [opt] <source_name> -o <exec_name>`

Exemple

- `gcc -Wall program.c -o program`
 - -Wall înseamnă *warnings=all*, adică se afișează toate warningurile
 - -Werror înseamnă *warnings=errors*, adică raportează warningurile

Executarea

- `./test`
- Necesită mai mulți pași
 - Alocarea unei zone de memorie pt spațiul de adresă
 - Încărcarea în memorie a executabilului
 - Configurarea regiștrilor din procesor cu valori specifice spațiului
 - Executarea efectivă a instrucțiunilor



Depanarea

Debugging – trasarea execuției pas cu pas – pentru a găsi erori (bugs)

- **Erori de compilare**

- Exemple: lipsă ; variabile nedeclarate tipuri incompatibile

- **Erori de runtime**

- Programul compilează dar crapă la execuție
 - Exemple: acces la memorie invalidă dereferențiere pointer NULL

- **Erori logice**

- Programul rulează dar rezultatul nu este corect

Unelte importante

- **`gdb`** (GNU Debugger)
- **Debugging din GUI** – metodă cunoscută
- **`printf debugging`** – metodă cunoscută
- **`valgrind`**

gdb

Compilare cu simboluri de debug

- `gcc -g program.c -o program`

Pornire debugger

- `gdb program`
 - Apare un shell interactiv

Comenzi utile

- `break main` – breakpoint
- `run` – rulează programul
- `next` – execută linia următoare
- `step` – intră în funcții
- `print x` – afișează valoarea variabilei
- `backtrace` – arată stiva de apeluri

Debugging post-mortem

Permite **examinarea stării** programului **din momentul crash-ului**

- analiza unui **core dump** – de regulă când programul face segmentation fault, abort, bus error
 - Fișier care conține
 - Memoria procesului
 - Regiștrii procesorului
 - Stiva
 - Valorile variabilelor

Activare core dump

- `ulimit -c unlimited`

Compilare cu simboluri de debug

- `gcc -g program.c -o program`

Rulare program și reproducere crash (core dumped)

- `./program`
- Dacă crapă -> se creează fișierul **core**

Analiza cu gdb – se încarcă fișierul împreună cu core dump-ul

- `gdb program core`

valgrind

Unealtă care detectează **memory leaks** și alte probleme de memorie - ține evidența memoriei alocate

Rezultatul: raport cu memoria rămasă nealocată

Compilare cu simboluri de debug

- `gcc -g program.c -o program`

Rulare

- `valgrind ./program argumente`

Opțiuni utile:

- `--leak-check=full` – arată detalii complete despre memory leaks
- `--show-leak-kinds=all` – arată toate tipurile de leak
- `--track-origins=yes` – arată de unde vine o eroare de memorie
- `-s` – rezumat la final
- Dacă se detectează leak-uri rulăm încă odată cu opțiuni:
- `valgrind --leak-check=full --show-leak-kinds=all -s ./program argumente`

Să ne reamintim...
limbajul C

Cuvinte cheie

Standardul C89			
auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Adăugate în standardul C99		
_Bool	_Complex	inline
	_Imaginary	restrict

Adăugate în standardul C11		
_Alignas	_Generic	_Thread_local
_Alignof	_Noreturn	
_Atomic	_Static_assert	

C17

- Bug fixes to C11

C23

- Ultimul standard

Comentarii

Recomandare

- La început comentariu cu datele de identificare
 - numele fișierului sursă, scopul programului, autorul și data scrierii respectiv a ultimei modificări

Comentariile pe mai multe linii pot urma stiluri diferite

```
/* Nume: suma.c
   Scop: calcul suma */
```

```
/******
 * Nume: suma.c      *
 * Scop: calcul suma *
 *****/
```

```
/* Nume: suma.c
 * Scop: calcul suma
 */
```

```
// Nume: suma.c
// Scop: calcul suma
```

Dezavantaj: simbolul de terminare este mai dificil de văzut.
- eroarea de omitere a simbolului de terminare este mai frecventă

Cadrul ajută la structurarea mai clară a programului
– delimitare mai bună

Elimina a trei dintre laturile cadrului din exemplul de mai sus
- mai puțin efort
- vizibilitate bună

Simbolul de început de comentariu trebuie repetat pe fiecare linie.
Elimină problema omiterii simbolului de terminare

Tipuri de date

Limbajul C are **cinci categorii fundamentale** de tipuri de date ([Link](#))

- tipul **întreg** – **int**: poate reține valori întregi, ex. 1, 0, -532, etc.;
- tipul **caracter** – **char**: poate reține un singur caracter sub forma codului elementelor din setul de caractere specific
 - codul **ASCII** (American Standard Code for Information Interchange) reprezentat pe 7 biți (poate reprezenta 128 de caractere) – set care este frecvent extins la codul **Latin-1**, pe 8 biți care poate reprezenta 256 de caractere
- tipul **real** (numere în virgulă mobilă) – **simplă precizie** – **float**: pot reține valori mai mari decât int și care conțin parte fracționară
 - de ex. 4971.185, -0.72561, etc.
- tipul **real** (numere în virgulă mobilă) în **dublă precizie** – **double**: pot reține valori reale în virgulă mobilă cu o precizie mai mare decât tipul float
- tipul **void**: indică lipsa unui tip anume. Un tip incomplet
 - Putem avea doar pointeri la void și funcții care returnează void (similar cu procedurile din alte limbaje)

Tipuri de date incluse începând cu C99

C99

- **long long**
 - Facilitează stocarea unor valori întregi de dimensiuni foarte mari - reprezentare pe cel puțin 8 octeți
 - Modificator de tip care se aplică doar asupra tipului **int**
- **Tipuri întregi de dimensiune fixă** ([Link](#))
 - Ex. **int8_t**, **int16_t**, **uint_32_t**, **int32_t**, **int64_t**, **intmax_t**, etc.
 - Valori întregi pe număr specificat de biți (definite în fișierul header **stdint.h**)
- **Tipul boolean** ([Link](#))
 - Tipul **_Bool** (De fapt valori întregi: 0 fals, orice altceva adevărat)
 - Macrodefiniții: **bool**, **true**, **false** (definite în fișierul header **stdbool.h**)
- **Tipuri complexe**
 - Tipurile **_Complex** și **_Imaginary**
 - Macrodefiniții: **complex**, **imaginary** (definite în fișierul header **complex.h**)
 - Ex. **float complex**, **double complex**, **long double complex**

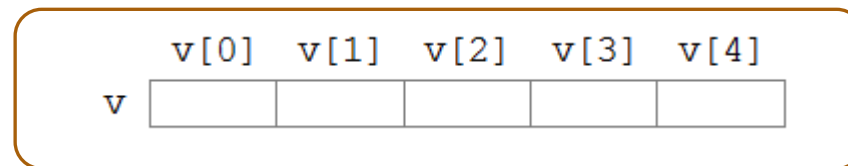
Tablouri unidimensionale - Vectori

Cel mai simplu tip de date agregat este *tabloul unidimensional*, numit și **vector**

Este o structură de date înrudite compusă din **valori de același tip** grupate sub un nume unic

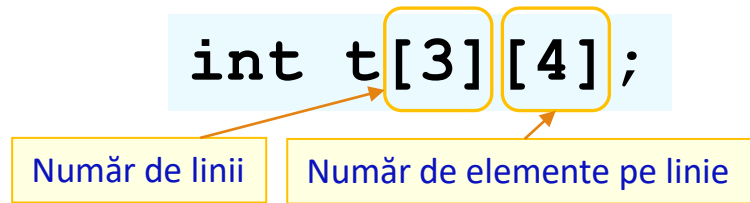
- Elementele tabloului
 - Acces prin index
 - Atenție: în C indexul începe de la 0

Declararea



```
int v[5];           // vector de intregi  
float distante[25]; // vector de valori reale  
char sir[100];      // vector de caractere
```

Tablouri bidimensionale – Matrice



Fiecare
element este
un vector

<code>t[0][0]</code>	<code>t[0][1]</code>	<code>t[0][2]</code>	<code>t[0][3]</code>
<code>t[1][0]</code>	<code>t[1][1]</code>	<code>t[1][2]</code>	<code>t[1][3]</code>
<code>t[2][0]</code>	<code>t[2][1]</code>	<code>t[2][2]</code>	<code>t[2][3]</code>

Inițializare la declarare

`int M[2][3] = { {1, 2, 3}, {4, 5, 6} };`

2 linii și 3 coloane

- Inițializarea **după** declarare implică instrucțiuni repetitive pentru a parcurge elementele iterative
- Accesul la elemente: prin procedeul de indexare

```
int sum;

M[1][2] = M[0][0] * 2;
sum = M[0][0] + M[0][1] + M[0][2] + M[1][0] + M[1][1] + M[1][2];
```

Funcții de intrare/ ieșire

Biblioteca standard de intrare/ieșire oferă mai multe categorii de funcții pentru efectuarea operațiilor de I/E

- disponibile prin includerea fișierului header **stdio.h**

- la nivel de **caracter**
 - Funcțiile getchar și putchar

- la nivel de **linie**
 - Funcțiile gets și puts

- cu **format**
 - Funcțiile scanf și printf
 - Specificatori de format

Atenție:
C11 a exclus funcția `gets()` din standard
- folosim `fgets()` în schimb

```
fgets(sir, 200, stdin);
```

- Dispozitivul standard de intrare – implicit tastatura
- Dispozitivul standard de ieșire – implicit ecranul monitorului

Ordinea de precedență și asociativitate

Ordine	Categorie	Operație	Operator	Asociativitate
1	Operatori postfixați	accesare element din tablou	[]	Stânga (stg -> dr)
		apel de funcție	()	
		accesare componentă din	. ->	
		structură		
		post-incrementare / post-decrementare	++ --	
2	Operatori unari	pre-incrementare / pre-decrementare	++ --	Dreapta (dr -> stg)
		semn plus	+	
		semn minus	-	
		negare logică	!	
		negare pe biți	~	
		adresă	&	
		dereferențiere	*	
		dimensiune	sizeof	
3	Conversie	conversie explicită	(tip)	Dreapta

Ordinea de precedență și asociativitate

Ordine	Categorie	Operație	Operator	Asociativitate
4	Operatori multiplicativi	înmulțire, împărțire, rest	* / %	Stânga
5	Operatori aditivi	adunare, scădere	+ -	Stânga
6	Operatori de deplasare	deplasare stânga, deplasare dreapta	<< >>	Stânga
7	Operatori relaționali	mai mic, mai mare, mai mic sau egal, mai mare sau egal	< > <= >=	Stânga
8	Operatori de egalitate	egal, diferit	== !=	Stânga
9	Operatori logici pe biți	și pe biți	&	Stânga
10		sau exclusiv pe biți	^	
11		sau pe biți		
12	Operatori logici	și logic	&&	Stânga
13		sau logic		Stânga

Ordinea de precedență și asociativitate

Ordine	Categorie	Operație	Operator	Asociativitate
14	Operator ternar	operatorul condițional expr_cond ? expr_da : expr_nu	? :	Dreapta
15	Operatori de atribuire	atribuire simplă, atribuire compusă aritmetică atribuire compusă pe biți	= += -= *= /= %= &= = ^= <<= >>=	Dreapta
16	Operator de secvențializare	operatorul virgulă	,	Stânga

Conversii implicite

Conversii implicite la atribuire

- Valoarea expresiei din dreapta se convertește la tipul expresiei din stânga
 - Probleme
 - Pot apare pierderi – dacă tipul nu este suficient de încăpător
 - Nu poate efectua conversia

Conversia implicită a tipurilor aritmetice

- conversia operanzilor se face la cel mai apropiat tip care poate reprezenta valorile ambilor operanzi

↑
long long
long
int
short
char
_Bool

Ierarhia tipurilor întregi standard

↑
long double
double
float

Ierarhia tipurilor reale standard

Reguli și recomandări

- Variabilele locale **trebuie să fie inițializate** explicit înainte de utilizare
 - Altfel pot conține valori neașteptate (valori reziduale ce se află în locațiile de memorie alocate variabilei)

```
#include <stdio.h>

int main()
{
    int i;
    printf("i: %d \n", i);

    return 0;
}
```

- Valoarea expresiilor trebuie să fie independentă de ordinea de evaluare a operanzilor – deci atenție la operanzii cu efecte secundare!

De evitat:

```
i = ++i + 1;
v[i++] = i;
a = i + v[++i];
```



```
i = i + 1;
a[i] = i;

++i;
a = i + b[i];
```

Corect:
Valorile expresiilor sunt
independente de ordinea de
evaluare a operanzilor

Reguli și recomandări

Folosirea parantezelor pentru evaluarea expresiilor compuse

- C are foarte mulți operatori - și regulile de precedență nu sunt întotdeauna intuitive
- Folosirea parantezelor adecvate poate reduce semnificativ riscul evaluării greșite a expresiilor
 - Facilitează înțelegerea și modificările ulterioare ale codului

Exemplu: Verificarea ultimului bit al variabilei x

Atenție:
== mai prioritar
decât &

x & 1 == 0



x & (1 == 0)



x & 0



(x & 1) == 0

Corect:

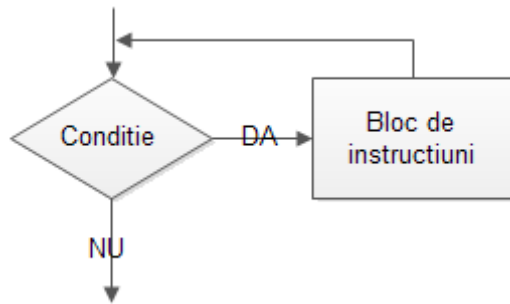
Parantezele asigură că evaluarea
se efectuează în modul așteptat

Excepție: expresiile algebrice

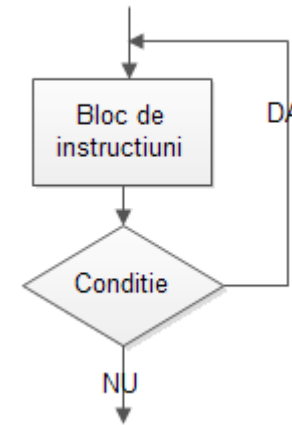
- Operatorii algebrici urmează cu strictețe ordinea cunoscută a operatorilor

Recapitulare instrucțiuni repetitive

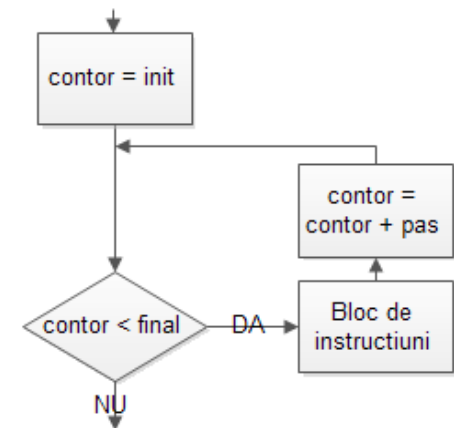
`while ( expresie ) instrucțiune`



`do instrucțiune while ( expresie );`



`for ( expresii_init ; expresie_control ; expresii_ajustare ) instrucțiune`



Instrucțiunile `break`, `continue` și `goto`

Realizează **salturi**

- întrerup controlului secvențial al programului și continuă execuția dintr-un alt punct al programului sau chiar provoacă ieșirea din program

Instrucțiunea `break` provoacă **ieșirea din instrucțiunea curentă**

Instrucțiunea `continue` provoacă **trecerea la iterația imediat următoare** în instrucțiunea repetitivă

Instrucțiunea `goto` produce un **salt la o etichetă predefinită** în cadrul aceleiași funcții

Reguli și recomandări

- Folosim acolade pentru corpul instrucțiunilor
 - chiar dacă este o singură instrucțiune în corp

În loc de:

```
for (i = 0, suma = 0; i < nr; i++)  
    suma += v[i];
```



```
for (i = 0, suma = 0; i < nr; i++) {  
    suma += v[i];  
}
```

Pentru

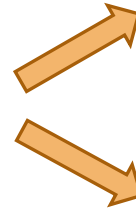
- **delimitarea și gruparea clară a instrucțiunilor**
- **fiabilitate la modificări**
 - Extindere ulterioară a corpului

Reguli și recomandări

- Să nu confundăm operatorul de atribuire (=) cu operatorul de egalitate (==) în contextele următoare
 - În expresia de control al instrucțiunilor if, while, do ... while,
 - Al doilea operand din for
 - Primul, al doilea sau al treilea operand din ? :
 - Oricare operand al operatorilor && și ||
 - Al doilea operand al operatorului ,

De evitat:

```
if (a = b) {  
    /* ... */  
}
```



```
if (a == b) {  
    /* ... */  
}
```

Corect:

Dacă se dorește testarea egalității,
și nu atribuire

```
if ((a = b) != 0) {  
    /* ... */  
}
```

Corect:

Dacă atribuirea este
intenționată

- Dacă unul din operanzi este o constantă, putem evita atribuirea nedorită prin plasarea constantei în partea stângă a operatorului

Eroare
sesizată
de compilator:

```
if (3 = a) {  
    /* ... */  
}
```



```
if (3 == a) {  
    /* ... */  
}
```

Corect:

Odată sesizată eroarea este remediată
simplu

Domeniul de vizibilitate

Un identificator (variabilă, constantă etc.) este accesibil **doar** în cadrul blocului în care a fost declarat.

```
int c = 30;
int main(){
    printf("%d\n", c);    // afiseaza 30
    {
        int a = 4;
        printf("%d\n", a); // afiseaza 4
        {
            int a = 5;
            char c = 'A';
            printf("%d\n", a); // afiseaza 5
            printf("%d\n", c); // afiseaza 65
                                // codul ASCII al lui 'A'
        }
    }
    printf("%d\n", c);    // afiseaza 30
    // a nu mai este definit aici
}
```

Variabila **a** din blocul exterior și variabila globală **c** sunt mascate

O variabilă declarată într-un bloc exterior este disponibilă și în blocul interior cu excepția cazului în care este redeclarată – caz în care variabila exterioară este “**mascată**” temporar.

Atenție: Mascarea poate îngreuna înțelegerea codului și depanarea.

Transmiterea parametrilor: prin valoare

În C, transmiterea parametrilor se face doar prin **valoare**

- Valoarea parametrului actual este **copiată** în parametrul formal corespunzător din funcția apelată

```
#include <stdio.h>
```

```
int suma_1_n(int n);
```

prototip

```
int main() {  
    int n, sum;
```

```
    printf("Introduceti un numar intreg: ");  
    scanf("%d", &n);
```

```
    sum = suma_1_n(n);  
    printf("Suma primelor %d numere \\  
           pozitive este: %d \n", n, sum);
```

```
    return 0;  
}
```

```
int suma_1_n(int n)  
{
```

```
    int suma = 0;  
    for ( ; n >= 0; suma += n, n--)  
        ;  
    return suma;  
}
```

Introduceti un numar intreg: 10
Suma primelor 10 numere pozitive este: 55

n rămâne neschimbat

**Copia locală a lui n,
independentă de n
din funcția apelantă**

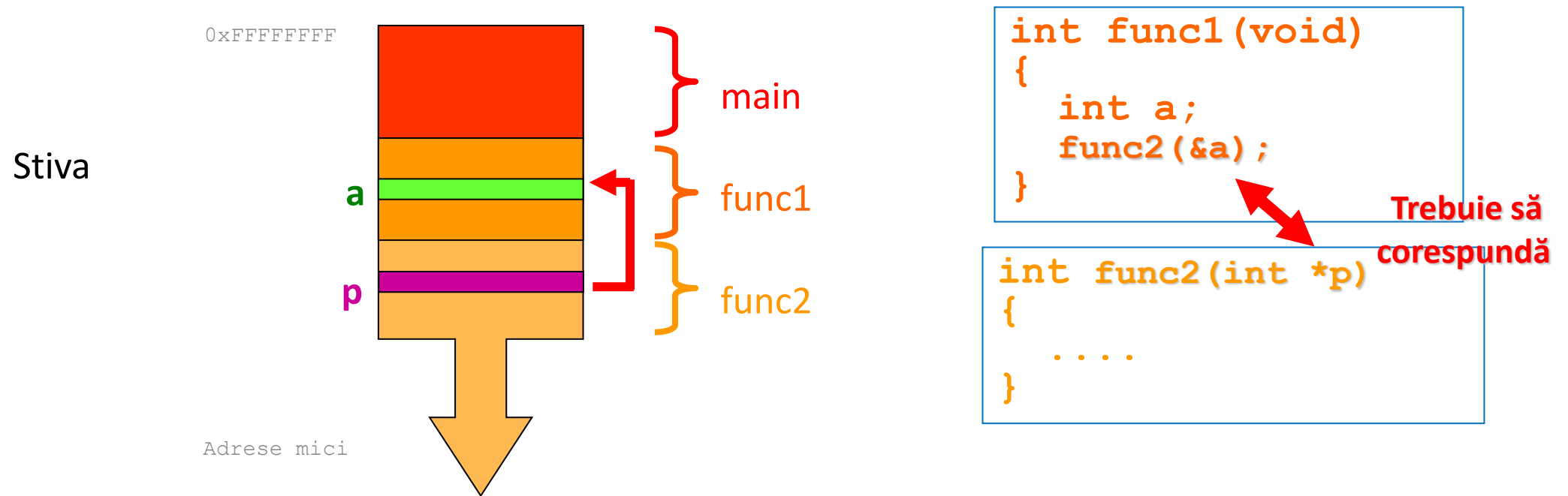
- Avantaj: protejează variabilele din funcția apelantă
- Dezavantaj: copierea poate fi ineficientă, ex. tablouri mari

→ pointeri

Transmiterea parametrilor

Cum ar putea o funcție să modifice valoarea argumentului primit, astfel încât modificarea să fie vizibilă în funcția apelantă?

→ pointeri



`func2` poate modifica `a` din `func1`, dacă are un pointer către `a` (cunoaște adresa lui `a`)

→ transitem `&a` (adresa variabilei `a`) ca argument funcției `func2`
(care la rândul său nu poate fi modificat de `func2` – adresa se transmite prin valoare)

Clase de stocare

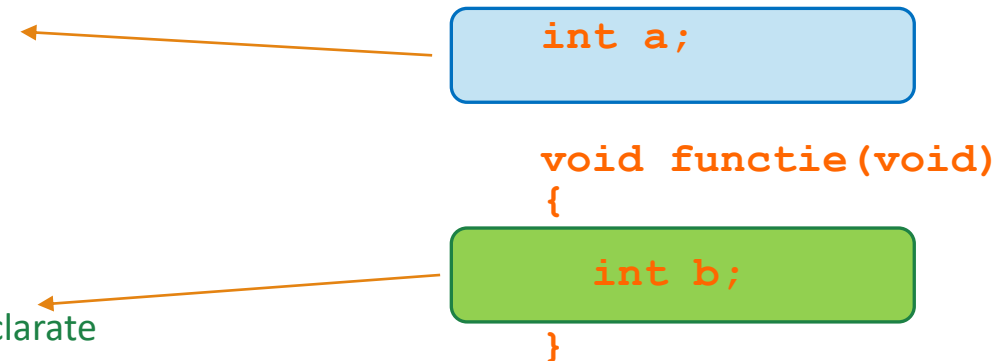
Valorile implicite ale acestor caracteristici depind de locația unde este definită variabila

- Variabilele declarate în exteriorul oricărui bloc (exteriorul funcțiilor) au

- Durata de stocare **statică**
- Vizibilitate în **întregul fișier** sursă
- Legare de tip **extern**

- Variabilele declarate în interiorul unui bloc de instrucțiuni (inclusiv corpul unei funcții sau al unei instrucțiuni) au

- Durata de stocare **automatică**
- Vizibilitatea doar în **cadrul blocului** unde au fost declarate
- **Nu** au legare



Caracteristicile implicite pot fi modificate folosind **clase de stocare** explicite

C furnizează 4 clase de stocare:

- **auto** **extern** **register** **static**

Reguli și recomandări

- Ordinea de evaluare a argumentelor transmise funcțiilor nu este specificată
 - Atenție la argumentele care în urma evaluării modifică valoarea altor argumente

```
extern void c(int i, int j);  
int glob;
```

```
int a(void) {  
    return glob + 10;  
}
```

```
int b(void) {  
    glob = 42;  
    return glob;  
}
```

De evitat:

```
void func(void) {  
    c( a(), b() );  
}
```

Nu știm ordinea în care
se evaluează argumentele funcției *c*

Valorile returnate de funcțiile *a()* și *b()*
depind de valoarea variabilei globale *glob*

Corect: ordinea este fixată

```
void func(void) {  
    int a_val, b_val;  
  
    a_val = a();  
    b_val = b();  
  
    c(a_val, b_val);  
}
```

Reguli și recomandări

O funcție nu poate avea ca tip returnat un tablou

```
int[] funcție(/*...*/) { /* ... */ }
```

Dar o funcție poate returna un pointer

```
int *funcție(/*... */) { /* ... */ }
```

Unul dintre argumentele funcției:
ex. pointer la un vector primit ca argument

```
int *cifre_nr(int v[], int n) {  
    int i = 0;  
    while (n != 0) { v[i] = n % 10; n = n / 10; i++; }  
    return v;  
}
```

Sau, mai frecvent:

```
int cifre_nr(int v[], int n) {  
    int i = 0;  
    while (n != 0) { v[i] = n % 10; n = n / 10; i++; }  
    return i;  
}
```

Elementele vectorului primit ca argument se modifică
și în funcția apelantă. Returnăm numărul de cifre

Pointer către zonă alocată dinamic în
funcție: ex. pointer la un vector alocat
dinamic

```
int *cifre_nr(int n) {  
    int i = 0;  
    int *v = (int *)malloc(100 * sizeof(int));  
    while (n != 0) {  
        v[i] = n % 10;  
        n = n / 10;  
        i++;  
    }  
    return v;  
}
```

Pointer către o variabilă locală statică:
ex. pointer la un vector alocat static

```
int *cifre_nr(int n) {  
    int i = 0;  
    static int v[100];  
    while (n != 0) {  
        v[i] = n % 10;  
        n = n / 10;  
        i++;  
    }  
    return v;  
}
```

Dar nu returnăm niciodată
pointer către variabilă automatică!!

```
int *cifre_nr(int n) {  
    int i = 0; int v[20];  
    while (n != 0) { v[i] = n % 10; n = n / 10; i++; }  
    return v;  
}
```

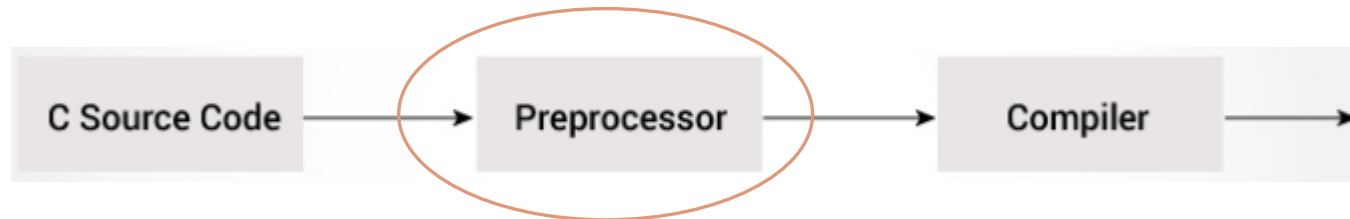
Preprocesorul

Preprocesorul este o unealtă care editează programele C înainte de compilare

- Analogia cu un editor de text: modifică și adaugă la codul sursă
- Poate introduce erori greu de depistat - codul poate deveni mai greu de înțeles -> folosim cu moderație

Etape de preprocesare se interpune între

- Editarea codului sursă și
- Compilare
 - Frecvent apare ca parte a procesului de compilare, și deci se efectuează transparent programatorului



Comportamentul preprocesorului este controlat de **directivele de preprocesare**

- Comenzi care încep cu # (diez)

Categorii de directive

- Definire de macrouri, definire de constante simbolice

`#define`

`#undef`

- Includere a fișierelor header

`#include`

- Compilare condiționată

`#if`

`#elif`

`#else`

`#ifdef`

`#ifndef`

`#endif`

- Altele

`#error`

`#line`

`#pragma`

Reguli și recomandări

Parametrii macrodefinițiilor se încadrează între paranteze rotunde pentru expandare corectă

- Cu excepția situațiilor în care parametri sunt despărțiți prin virgule sau când se dorește concatenarea sau convertirea la șiruri a parametrilor

```
#define CUBE(X) ( X * X * X )  
  
void func(void) {  
    int a = 81 / CUBE(2 + 1);  
    /* ... */  
}
```



```
#define CUBE(X) ( (X) * (X) * (X) )  
  
void func(void) {  
    int a = 81 / CUBE(2 + 1);  
    /* ... */  
}
```

După expandare:

```
int a = 81 / (2 + 1 * 2 + 1 * 2 + 1);  
/* Se evalueaza la 11 */
```

După expandare:

```
int a = 81 / ( (2 + 1) * (2 + 1) * (2 + 1) );  
/* Se evalueaza la 3 */
```

Reguli și recomandări

Textul de substituție se introduce între paranteze rotunde pentru expandare corectă

- Pentru a trata valoarea macrodefiniției ca un singur operand în expresiile care apare

```
#define CUBE(X) (X) * (X) * (X)

void func(void) {
    int i = 3;
    int a = 81 / CUBE(i);
    /* ... */
}
```



```
#define CUBE(X) ( (X) * (X) * (X) )

void func(void) {
    int i = 3;
    int a = 81 / CUBE(i);
    /* ... */
}
```

După expandare:

```
int a = 81 / i * i * i;
/* Echivalent cu int a = (((81 / i) * i) * i); */
/* Se evalueaza la 243 */
```

După expandare:

```
int a = 81 / ( (3) * (3) * (3) );
/* Se evalueaza la 3 */
```

Reguli și recomandări

Se recomandă folosirea funcțiilor `inline` în locul macrodefinițiilor

- Macrodefinițiile sunt substituții textuale și pot introduce erori sau comportament nedefinit – mai ales dacă parametrul are efecte secundare

```
#define CUBE(X) ((X) * (X) * (X))

void func(void) {
    int i = 2;
    int a = 81 / CUBE(++i);
    /* ... */
}
```



```
inline int cube(int i) {
    return i * i * i;
}

void func(void) {
    int i = 2;
    int a = 81 / cube(++i);
    /* ... */
}
```

După expandare:

```
int a = 81 / ((++i) * (++i) * (++i));
```

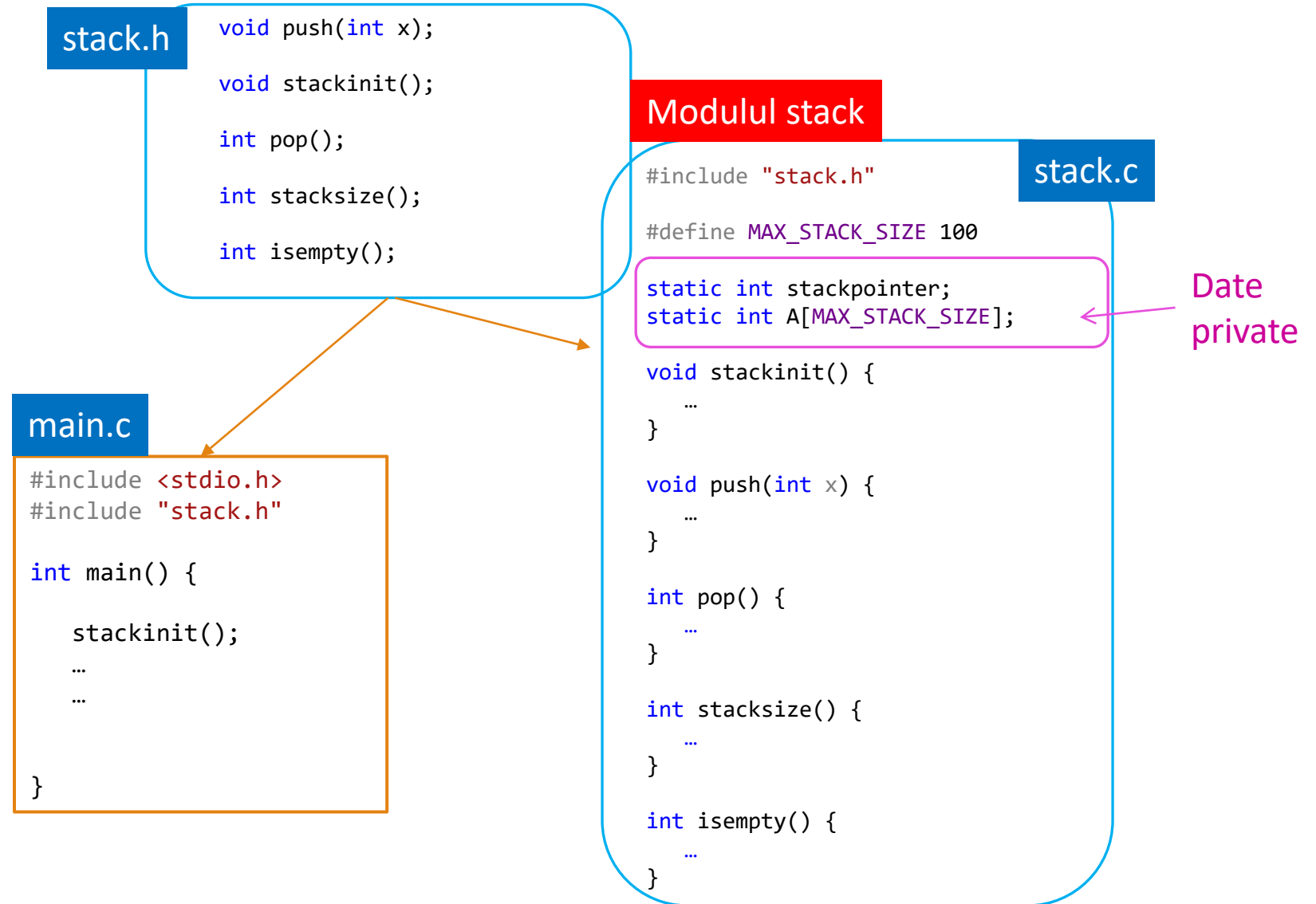
Folosind funcție `inline` în locul macrodefiniției, expresia cu efect secundar este evaluată o singură dată, înaintea apelului de funcție

Cum arată un Modul?

Un modul poate fi împărțit în două entități

- **Interfața** - fișiere header **.h**
 - Este **publică**
 - Arată cum se pot apela funcțiile din modul, care sunt tipurile de date definite de modul
 - Conține
 - Declarații de funcții (prototipuri)
 - Definiții de constante simbolice (#define)
 - Definiții sau declarații de tipuri de date
 - Informații care **pot fi referite** din afara moduluilui
 - Dacă se include fișierul header în program (#include)
- **Implementarea** - fișiere sursă **.c**
 - Este **privată**
 - Informații interne, **implementarea** specifică
 - Definiții de tipuri și funcții
 - Informații care **NU pot fi referite direct** din afara moduluilui

Exemplu



Recomandări

Protejarea împotriva includerii multiple a aceluiași fișier header

- Soluție
 - Conținutul fișierului header se include între directive de compilare condiționată

stack.h

```
#ifndef __STACK_H_
#define __STACK_H_

void push(int x);

void stackinit();

int pop();

int stacksize();

int isempty();

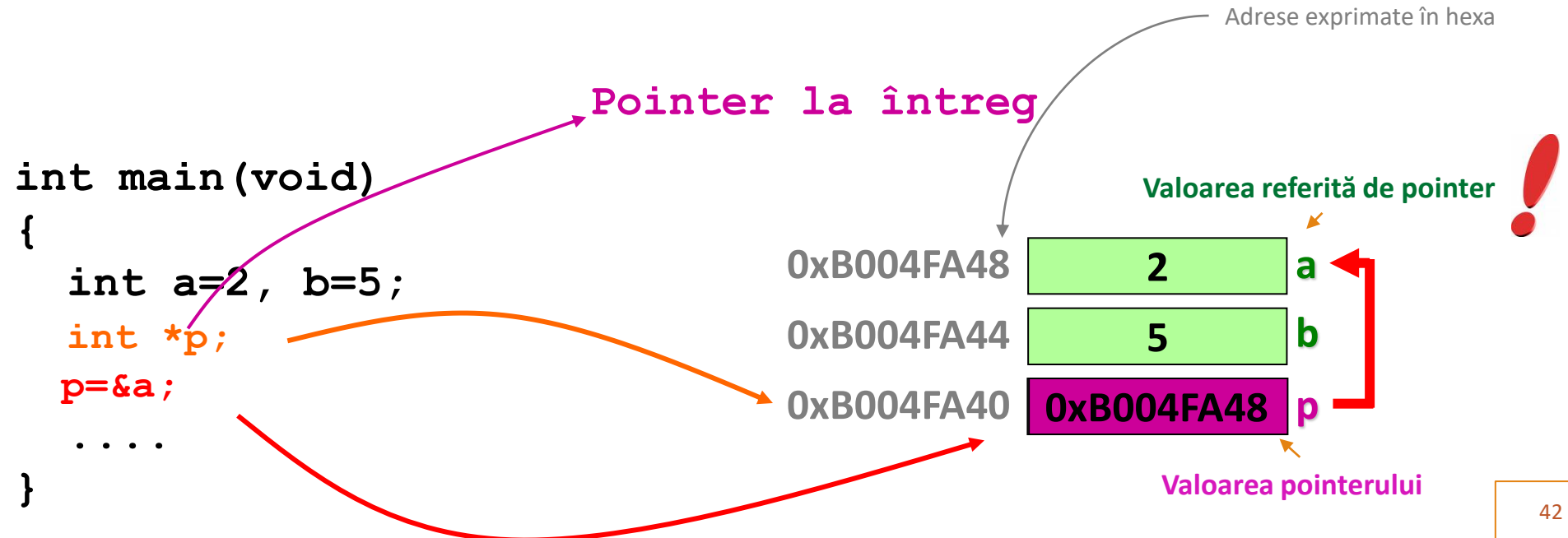
#endif
```

Pointeri

Declarare

- Asemănător cu variabilele "obișnuite", dar se pune o stelută (asterisc) înaintea identificatorului
 - Tipul indică **tipul referit**
 - Adică faptul că variabila pointer poate stoca adresa unui obiect (variabilă) de tipul specificat
- Limbajul C impune ca fiecare pointer să indice doar la variabile (obiecte) de un anumit tip
 - Nu sunt restricții asupra tipului

tip *p;



Tablouri și pointeri

Operatorul adresă poate fi folosit și asupra elementelor tablourilor

```
int a[10], *p;  
p = &a[0];    // p pointează către primul element  
p = &a[1];    // ... către al doilea element
```

Soluție mai simplă:

În C, **numele tabloului este pointer la începutul tabloului** !

<code>p = a;</code>	este echivalent cu	<code>p = &a[0];</code>
<code>p = a+1;</code>	este echivalent cu	<code>p = &a[1];</code>

! Există totuși o diferență între **a** și **p**:
a este constant (fixat), deci ~~`a = p;`~~ este ilegal!

```
int a[10], *p, i;
```

`a[i]` este echivalent cu `*(a + i);`

`p[i]` este echivalent cu `*(p + i);`

**Aritmetica
pointerilor**

Pointer la void

Pointerii sunt legați de **tipul** către care pointează. Excepție: pointeri generici

void * este un pointer generic

- poate fi legat ulterior la orice tip de variabilă
- folosind conversia explicită (type cast) la tipul corespunzător

Exemplu:

```
int    *p;  
void   *v;
```

Dacă v pointează la un întreg, atribuirea corectă se face astfel:

```
p = (int *) v;
```

Ne asigurăm că tipurile pointerilor coincid.

Exemplu:

```
int    i;  
void   *v;
```

i = 5; // este echivalent cu

```
v = &i;  
*(int *)v = 5;
```

Dereferențierea nu poate fi făcută
fără conversie explicită (type cast) !

```
int inx = 376;  
double dnx = 3.14;  
void *ptr = &inx;  
...  
printf( "%d\n", *(int *)ptr );  
ptr = &dnx;  
printf( "%f\n", *(double *)ptr );
```

Erori frecvente

1. Dereferențierea unui pointer neinițializat

```
int *p;  
*p=5;
```

2. Referință invalidă

```
int a,*p, *q;  
p=&a;  
p=q;
```

3. Referință nulă

```
int *p;  
p=NULL;  
*p=5;
```

Pointeri - Exemplan

```
int v[20] = {3, 7, 9, 11, 16};
int *p;
int i;

for (i=5; i< 10; i++)
    scanf("%d", v+i);

for (p=v+10; p-v < 20; p++)
    scanf("%d", p);

p = v;      /** p = &v[0]; */
i = *p++;   /** p = &v[1], i = 3 */
i = *++p;   /** p = &v[2], i = 9 */
i = ++*p;   /** p = &v[2], v[2] = 9+1, i = 10 */
i = p-v;    /** (&v[2] - &v[0]) / sizeof(int), i = 2 */
```

Pointeri - Exemplu

```
int v[20] = {3, 7, 9, 11, 16};
int *p;
int i;

for (i=5; i< 10; i++)
    scanf("%d", v+i);

for (p=v+10; p-v < 20; p++)
    scanf("%d", p);

p = v;          /** p = &v[0]; */
i = *p++;       /** p = &v[1], i = 3 */
i = *++p;       /** p = &v[2], i = 9 */
i = ++*p;       /** p = &v[2], v[2] = 9+1, i = 10 */
i = p-v;        /** (&v[2] - &v[0]) / sizeof(int), i = 2 */
```

v este un pointer constant

Primele 5 elemente se inițializează explicit - restul implicit devin 0

Notăția specifică pointerilor
Aritmetica pointerilor

Parcurgem elementele (v[10] - v[19]) prin intermediul pointerului variabil p

Postincrementarea este mai prioritară decât dereferențierea
p=p+1; // se returnează valoarea neincrementată
i=*p;

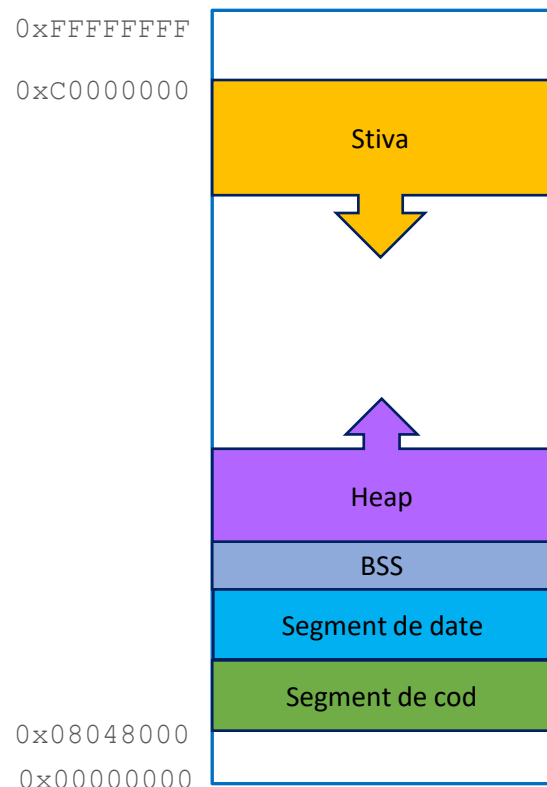
Aritmetica pointerilor
Câți întregi încap între p și v ?

p=p+1; i=*p;

*p=*p+1; i=*p;

Funcții pentru alocarea dinamică a memoriei

stdlib.h



Exemplificare pentru alocarea dinamică a unui vector de n întregi

calloc vine de la "contiguous allocation"

Inițializează
spațiul alocat cu 0

```
p = (int *) calloc(n, sizeof(int));
```

Pointerul returnat conține adresa de început a spațiului alocat (sau NULL în caz de eroare).

Pointerul returnat este generic (pointer la void) -> este necesară conversia explicită (cast) la tipul dorit!

Tipul `int` din exemplul nostru trebuie înlocuit cu tipul dorit

Alternative:

```
p = (int *) malloc(n * sizeof(int));
```

Fără inițializare

Un singur argument: dimensiunea totală

```
ptr = (int *) realloc(p, dimensiune_noua);
```

Schimbă dimensiunea spațiului la care pointează `p` la `dimensiune_noua`

Eliberarea memoriei:

```
free(p);  
p = NULL;
```

Trebuie dealocat explicit

Dacă `p` este NULL, atunci `free` nu va avea nici un efect

Atenție: după `free()` `p` pointează către o adresă invalidă, deci asignăm NULL

Funcții pentru alocarea dinamică a memoriei

Exemplu folosind realloc

```
ip = realloc(ip, sizeof(ip) + sizeof(int)*5);
```

Așa nu



Ce se întâmplă dacă eșuează realloc?

ip va deveni NULL

Nu returnează dimensiunea spațiului alocat inițial

Spațiul alocat precedent nu va putea fi accesat!!!

```
int *tmp = realloc(ip, (N + 5) * sizeof(int));
if ( tmp == NULL)
{ /* În caz de eșec se poate elibera ip...
   Sau alte acțiuni potrivite scopului propus */
  fprintf(stderr, "ERROR: realloc failed");
}
ip = tmp;
```

Reguli și recomandări

Nu aplicăm operatorul `sizeof` pe un pointer pentru aflarea dimensiunii tabloului

- Operatorul `sizeof` determină numărul de octeți ocupați de operandul dat
 - Operandul poate fi o variabilă, un tip, o expresie
- Determinarea dimensiunii unui tablou folosind operatorul `sizeof` poate introduce erori – referirea la tablou poate fi un simplu pointer

Exemplu:

Asignarea valorii 0 la fiecare element din vector

Parametrul `array` este doar un pointer, nu un vector

```
void clear(int array[]) {  
    for (size_t i = 0; i < sizeof(array) / sizeof(array[0]); ++i) {  
        array[i] = 0;  
    }  
}  
  
void dowork(void) {  
    int dis[12];  
  
    clear(dis);  
    /* ... */  
}
```

```
void clear(int array[], size_t len) {  
    for (size_t i = 0; i < len; i++) {  
        array[i] = 0;  
    }  
}  
  
void dowork(void) {  
    int dis[12];  
  
    clear(dis, sizeof(dis) / sizeof(dis[0]));  
    /* ... */  
}
```

Corect:

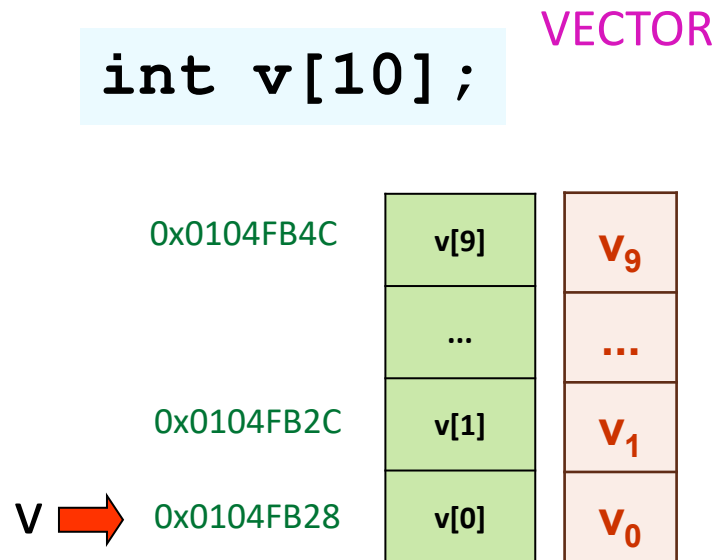
Dimensiunea este calculată și trimisă funcției ca parametru

- Calculul va fi corect având definiția vectorului `dis`

Tablouri unidimensionale vs. bidimensionale

Memoria este adresată liniar (este similar unui vector foarte-foarte lung)

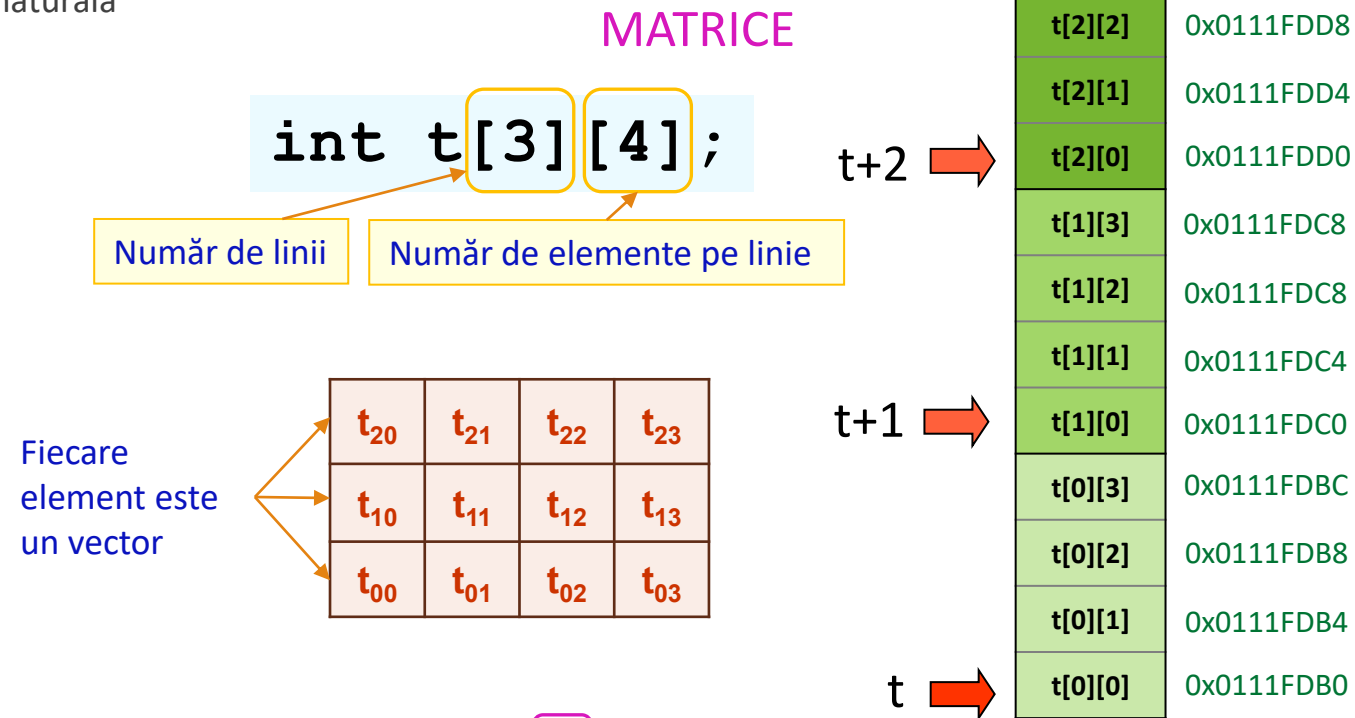
- Reprezentarea tablourilor unidimensionale este foarte naturală



v este adresa primului element **v** este &v[0]
 pointer constant la întreg

Accesul la elemente:

Elementul de la poziția i: **v[i]** sau ***(v+i)**



t este adresa primului element **t** este &t[0]
 pointer constant la un vector de 4 întregi

t[0] este &t[0][0]
 pointer constant la întreg

Accesul la elemente:

Elementul de pe linia i poziția j: **t[i][j]** sau ***(*(t+i)+j)**
 sau ***(t[i]+j)**
 sau ***(t+i)[j]**

Tablouri bidimensionale – tablou de pointeri

În matrice fiecare element este un vector

- Fiecare vector este un pointer constant la întreg



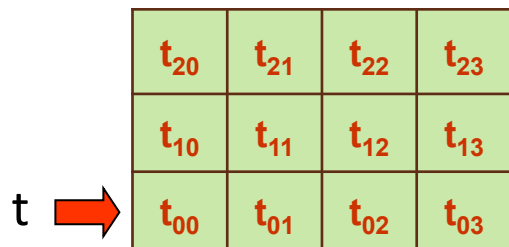
Tabloul bidimensional poate fi reprezentat și de un **tablou de pointeri**

Tabloul de pointeri și pointerul la pointer permit a gestiune mai flexibilă a memoriei

Matrice

```
int t[N][M];
```

```
int t[3][4];
```



S-a alocat **spațiu pentru NxM întregi**

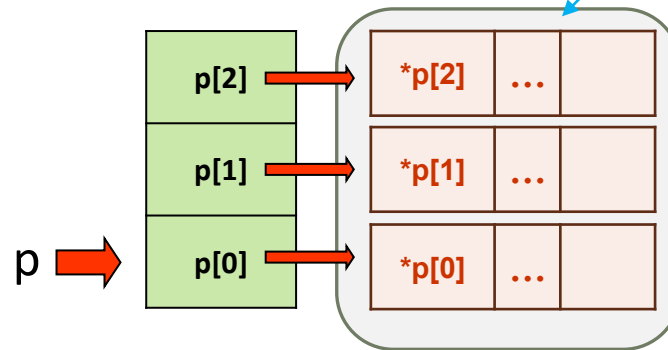
- t este un pointer constant la primul element

t este &t[0][0]

Vector de pointeri

```
int *p[N];
```

```
int *p[3];
```



S-a alocat spațiu pentru **N pointeri la întregi**

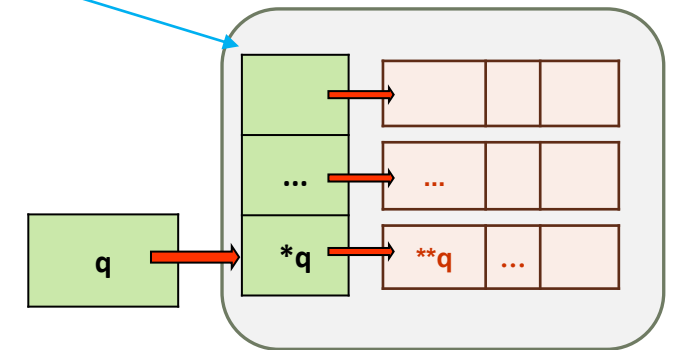
- p este un pointer constant la primul element

p este &p[0]

Pointer la pointer

```
int **q;
```

```
int **q;
```



S-a alocat spațiu pentru un **pointer care indică un pointer la întreg**

- q este un pointer variabil

Tablouri neregulate

Un tablou de pointeri care indică vectori de dimensiuni diferite se numește un **tablou neregulat**

Matrice

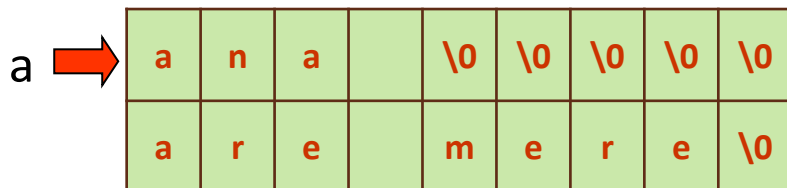
```
char a[2][9] = { "ana ", "are mere" };
```

```
printf("%c%c%c... %s %s\n", a[0][0],  
*(a[0] + 1), a[0][2], a[0], a[1]);
```

Rezultat: ana... ana are mere

a este o matrice => se alocă explicit spațiu pentru 2x9 = 18 caractere

- Doar primele 5 caractere sunt utilizate 'a', 'n', 'a', ' ' și '\0' în a[0]
- restul sunt inițializate implicit la 0



Vector de pointeri

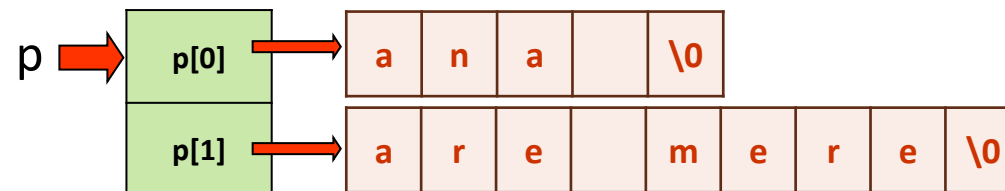
```
char *p[2] = { "ana ", "are mere" };
```

```
printf("%c%c%c... %s %s\n", **p,  
p[0][1], *((*p) + 2), p[0], p + 1);
```

Rezultat: ana... ana are mere

p este un vector de pointeri => se alocă explicit spațiu pentru 2 pointeri

- p[0] este inițializat să poarteze către "ana ", un șir constant de 5 caractere
- p[1] este inițializat să poarteze către "are mere" care necesită 9 caractere. Șirurile constante sunt alocate implicit în segmentul de date



Stringuri

În C stringurile sunt implementate ca și **tablouri unidimensionale** de tipul **char**, având ultimul element caracterul nul **'\0'**

- Caracterul nul (caracterul cu codul 0) are rolul de a indica sfârșitul șirului

Nu confundați

caracterul nul (`'\0'`) – cu codul 0 (toți biții din reprezentare sunt 0) și

caracterul zero (`'0'`) – caracterul dinaintea caracterului `'1'` (având codul ASCII 48)

Atenție: trebuie alocat spațiu
și pentru caracterul nul !!

```
char fruct[] = { 'p', 'e', 'p', 'e', 'n', 'e', ' ', 'g', 'a', 'l', 'b', 'e', 'n', '\0' };
```

Șir de caractere variabil

Inițializare mai practică

```
char fruct[] = "pepene galben";
```

`\0` se adaugă automat
(nu trebuie specificat explicit)

```
char *fruct_p = "pepene galben";
```

Șir de caractere constant

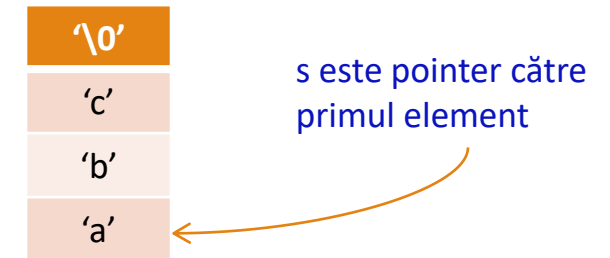
Stringuri

Numele șirului de caractere este pointer la primul element

- Accesul la șir se face prin intermediul acestui pointerului la primul element

```
char sir[10]; // sir este &sir[0]
```

```
char s[ ] = "abc";    printf("%s %s\n", s, s+1);  /* se afișează abc bc */
```



Diferența dintre un caracter și un șir de caractere

- "a" este un șir de caractere - un tablou de 2 caractere
- 'a' este un caracter



Există o diferență fundamentală între șirurile definite ca și **tablouri de caractere** (pentru care se alocă spațiu explicit) și **șirurile constante**, scrise între ghilimele, ex. "abc" pentru care nu s-a alocat explicit spațiu

```
char s[ ] = "abc";
```

s este pointer constant către un șir variabil
(spațiu de 4 caractere alocat explicit)

```
printf("abc");
```

"abc" este un șir de caractere constant
(nu s-a alocat explicit spațiu)

Recapitulare: șiruri de caractere în C

	Șiruri constante	Tablou de caractere	Șiruri alocate dinamic
Memoria	Segm. de date	Stiva	Heap
Declarare simplă	Trebuie inițializat	<code>char s[4];</code>	<code>char *s=(char *) malloc(4);</code>
Declarare cu inițializare	<code>char *s ="abc";</code>	<code>char s[] ="abc";</code>	<code>char *s=(char*) calloc(4,1);</code> (inițializat cu \0)
Pointer	În fiecare caz s este pointer către primul caracter din șir		
Atribuirea unui caracter	Nepermis	<code>s[1]='B';</code> <code>*(s+2)='C';</code>	acestea se folosesc de funcțiile de mai jos ↓
Atribuirea unui șir	Nepermis	<code>strcpy(s,"ABC");</code> <code>strcpy(s,s2);</code> <code>sprintf(s,"%s%d%c","a",2,'c');</code> <code>scanf("%3s",s);</code> (de la intrare) <code>sscanf("ABC","%3s",s);</code>	
Reassignarea pointerului	Se pierde conținutul	fixat (<u>nu poate fi</u> reassignat)	Se pierde referința la memorie
Eliberarea memoriei	Nu	La ieșire din { } bloc sau funcție	<code>free(s);</code>

Reguli și recomandări

Copierea directă a șirurilor folosind operatorul de atribuire (=) **nu este permisă**

```
char s1[25], s2[25];
```

```
s1 = "abcd";  
s2 = s1;
```

Greșit!

Numele șirului nu poate apare în
partea stângă a unei atribuirii

Folosind funcțiile din
string.h



Inițializarea șirului la declarare **este permisă**

```
char s[] = "abcd";
```



Atenție!

NU folosim funcții pe stringuri
dacă nu suntem absolut siguri
că se termină cu **caracterul nul**

Compararea șirurilor de caractere nu se face folosind operatori relaționali sau de egalitate

- Operația este permisă, dar va compara valorile pointerilor și nu conținutul șirurilor de caractere

```
if (s1 == s2) ...
```

Greșit!

Argumentele funcției main()

Funcția principală poate primi cel puțin două argumente la apelarea programului
Acele argumente au numele convențional `argc` și `argv`.

argc reține **numărul argumentelor** (primul fiind numele programului),

argv este un **tablou de pointeri la caracter** (listă de cuvinte)
care arată către fiecare argument în parte.

Adesea numele fișierelor sunt transmise în acest fel.

Exemplu: `afis_param.c`

```
/* Afisarea argumentelor primite */
#include <stdio.h>
int main(int argc, char *argv[]) {
    int i;

    printf("argc = %d\n", argc);
    for (i = 0; i < argc; ++i)
        printf("argv[%d] = %s\n", i, argv[i]);

    return 0;
}
```

argc este numărul argumentelor

argv[i] sunt argumentele

`afis_param fis1 fis2`
atunci **argc** este 3;
argv[0] este `afis_param`
argv[1] este `fis1`
argv[2] este `fis2`

Argumentele funcției main()

Argumente numerice

Argumentele sunt preluate ca șiruri de caractere → argumentele numerice trebuie convertite

atoi() pentru conversia șirului la întreg
atof() pentru conversia unui șir în double

Exemplu:

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int i;
    double x;

    i = atoi(argv[1]);
    x = atof(argv[2]);

    printf("Input = %d, %lf\n", i, x);

    return 0;
}
```

sscanf(argv[1], "%d", &i);

sscanf(argv[2], "%lf", &x);

Structuri

Accesul la componente – prin procedeul de **calificare**

```
typedef struct {  
    long cod;  
    char nume[50];  
    float pret;  
} produs ;  
produs p1, p2;
```

```
p1.cod = 325671;  
strcpy(p1.nume, "Caiet");  
p1.pret = 1.4;
```

Atribuirea – este validă **p2 = p1;**

Ne reamintim: La tablouri atribuirea nu este validă !!

Compararea – NU este validă



```
p2 == p1; /* compararea NU este validă */  
p2 != p1; /* compararea NU este validă */
```

Inițializarea – cu valori potrivite fiecărei componente

```
produs p3 = {42315, "Stilou", 34.9f};  
produs p2 = { .cod = 24385, .pret = 54.0f };
```

Se pot folosi inițializatori desemnați – C99

Tablouri de structuri – uni- sau multi-dimensionale

```
produs raft[25];
```

Declară un vector de 25 de elemente de tip produs
- numele tabloului este raft.

Actualizarea structurii în funcție

Transmitem structura *e* funcției, o modificăm, și o returnăm:

```
e = modifica1(e);
.....

date_angajat modifica1(date_angajat e)
{
    ....
    printf ("Introduceti numarul departamentului: ");
    scanf ("%d", &n);
    /* accesam componenta structura... */
    e.departament.nr_depart = n;
    ....
    return e;
}
```

Implică atât copierea componentelor structurii în parametrul formal *e* cât și copierea rezultatului înapoi în variabilă *e* din funcția apelantă.

Există o soluție mai bună... Transmitem pointer la structura

Transmiterea pointerului la structură este o soluție mult mai eficientă:

```
modifica2(&e);
.....

void modifica2(date_angajat *e)
{
    ....
    printf ("Introduceti numarul departamentului: ");
    scanf ("%d", &n);
    /* accesam componenta structura... */
    e->departament.nr_depart = n;
    ....
    return e;
}
```

Apelăm funcția și transmitem adresa structurii *e*

Folosim *->* pentru accesul la componente

Sugestii și recomandări

Procesul de scriere a programelor în limbajul C este unul **iterativ** și **incremental**

implică testare continuă,
identificarea și corectarea erorilor,
precum și optimizarea ulterioară a codului

Editor sau IDE?

IDE-ul oferă multe facilități -> productivitate mare
Editorul permite învățare mai profundă

Programare defensivă

anticiparea posibilelor probleme și
introducerea de verificări pentru a preveni erorile sau comportamentele neprevăzute
verificarea input-ului și valorilor returnate, validarea parametrilor, verificarea limitelor,
tratarea erorilor, etc.